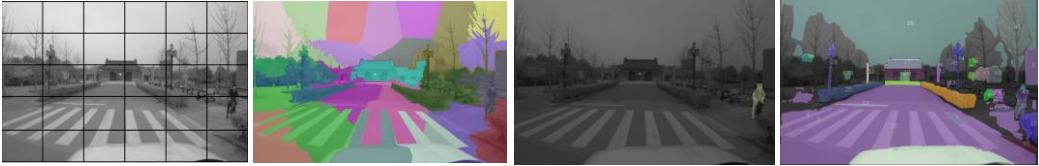


图像分割算法

为了有效降低故障模式总数 $N_{subsets}$ ，本节引入图像分割（Image Segmentation）技术，将原始的特征点量测根据其在图像空间中的分布特性及语义相关性进行聚合，改进量测形式。分组的核心逻辑在于：在动态车载环境下，视觉跟踪故障往往不是孤立发生的，而是具有显著的空间相关性。例如，由于遮挡、局部光照剧变或动态物体干扰（如前方大型车辆经过）导致的跟踪失败，通常会同时影响图像中某一局部区域内的所有特征点。

本节对多种主流图像分割技术进行了对比研究，旨在寻找最适合车载导航环境的特征点分组方案。如所示，本研究针对车载导航环境，对比了四种具有代表性的图像分割方法：

	均匀分割	聚类分割	YOLO-seg	SAM
来源			Ultralytics	Meta AI
训练			√	√
分割一切				√
实例分割			√	√
具体模型	Grid	SLIC（2010）	Yolo11n-seg（2024）	FastSAM（2023）
大小（MB）			5.9	23.7
参数（M）			2.9	11.8
速度（ms/im)*	2-3	171-180	35-47	78-113
特点	速度快	边界不精确，过分割	只能分割训练过类别	通用性强，速度慢
分割举例				

- 均匀分割（Grid Segmentation）：通过简单的 5×6 或 4×4 规则网格对图像进行划分。其优势在于处理速度极快（约 2-3 ms/im），但缺陷在于完全忽略了场景的几何结构和语义逻辑，容易将属于同一物理实体的特征点切割到不同组中，导致故障假设与实际风险源脱节。
- 聚类分割（SLIC）：基于局部 k-means 思想生成超像素。虽然能根据颜色和位置相似度进行初步聚合，但其算法依赖多轮迭代，处理速度较慢（约 171-180 ms/im，且边缘划分往往不够精确，存在严重的过分割现象）。
- 目标检测分割（YOLO-seg）：以 Ultralytics 推出的 YOLO11n-seg 为代表，推理速度较快（约 35-47 ms/im）。然而，该方法受限于训练集的类别定义，只能分割预定义的特定目标（如行人、车辆），对于城市环境中的非结构化背景（如形态各异的建

筑物、树木) 缺乏泛化能力。

- 全分割模型 (SAM / FastSAM): 由 Meta AI 提出的 Segment Anything Model (SAM) 具备“分割一切”的强大通用性。针对其速度较慢的问题, 本研究选用了由中科院等团队提出的 FastSAM (2023)。该模型在保留 SAM 强通用性的同时, 通过参数压缩显著提升了推理速度。实验表明, FastSAM 能够有效分割天空、路面、汽车盖、骑行者以及复杂的交大东大门结构, 使图像中的每一个视觉量测都能被赋予合理的归属

考虑到实时性需求, 本研究最终采用 FastSAM (2023) 作为基础模型, 但尽管 FastSAM 在模型规模上已大幅精简, 但直接应用于高频导航系统时, 其原生输出仍存在掩码细碎、重叠以及计算耗时波动大的问题。如**错误!未找到引用源。**所示, 本研究提出了一套针对性的改进方案:

(1) 后处理:

- ① 面积优先的层级归属: 针对多层掩码重叠的现象, 按照区域面积大小建立层级, 确保小目标不会被大背景淹没, 使特征点能够准确挂载到最细分的物理层级上。
- ② 最大连通域分析: 对分割生成的 Mask 进行连通性检查, 滤除散落的孤岛像素噪声, 确保每一个故障组在物理空间上是连续且完整的。
- ③ 小目标过滤与目标剪枝: 针对细微的分割块进行剪枝, 将其合并至相邻的大组中。这一步骤能直接压缩独立故障源总数, 避免由于分组过多导致的 $N_{subsets}$ 再次膨胀。

(2) 推理加速:

- ① ONNX Runtime 部署: 通过将 PyTorch 模型转换为 ONNX 格式, 利用硬件底层的并行加速能力。
- ② 输入空间降维: 对原始高分辨率图像进行适度的空间降维预处理。

经过上述优化, 图像分割算法在保持高精度分割的同时, 单帧耗时从 100ms 以上缩减至 39-58ms, 成功匹配了车载融合导航系统的运行节奏。

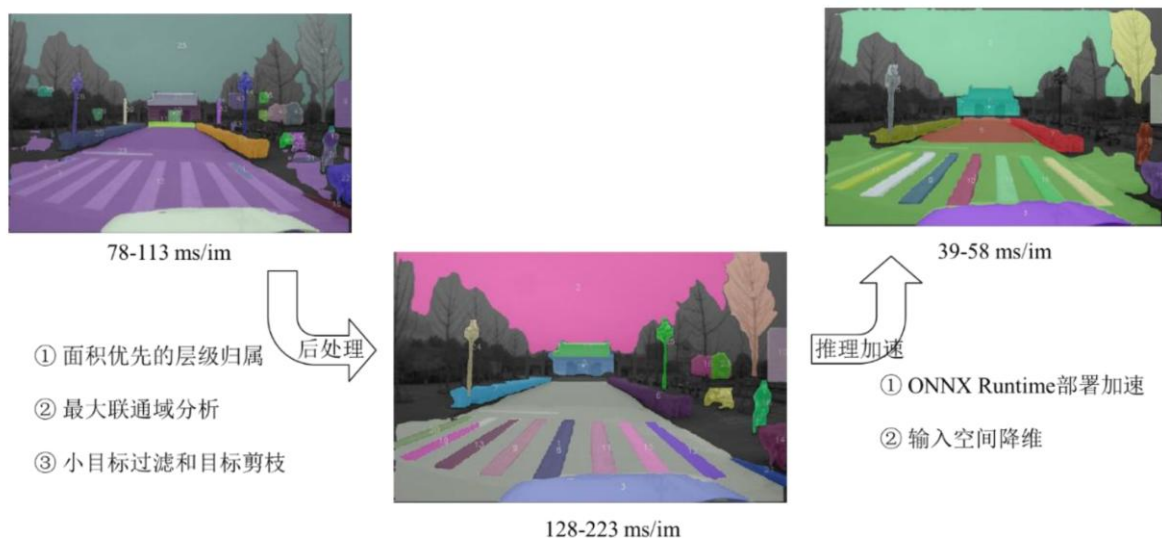


图4-8 FastSAM 的算法改进方案
Fig. 4-8 Algorithm improvement of FastSAM

文字部分拟从以下几点描述（红色为请求帮助部分）

总思路：原先并非 **corner-oriented** 的，作了细化。但同时注意抑制细化带来的副作用

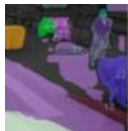
【请求：1、可以三个 **challenge** 合在一起搞个伪代码（每个 **challenge** 的 **solution** 要有一定展开），同时三个 **challenge** 各有一篇 **ref** 说明 **baseline** 代码或改进思路来源

或者 2、分别回答三个 **challenge**，同时三个 **challenge** 个各有一篇 **ref** 说明 **baseline** 代码或改进思路来源

】

reproducible 是论文写作的基本要求，所以要求这些细节。

	challenge	solution	龙答
1	not enough resolution in corner-rich regions: only entire object (road), but we need smaller detail	面积优先的层级归属：小目标不会被大背景淹没 【问孙宇龙要 ref	

	(stripes of zebra crossing on the road) 例：横道线	指向 algorithm / 直接给出伪代码+ threshold 。找到 corner 。	
2	too high resolution in corner- barren regions: too many closely isolated meaningless patches; 例：4-8 最左图，最右下角 	最大 连通域分析 【问孙宇龙要 ref 】：滤除孤岛（像素级）。 subset number reduction 第一步 附加效果： 物理可解释	
3	too many closely isolated patches although feature-rich, error- prone corner detection 例：4-8 最左图，右边树下区域 	剪除过小块，第 2 步并非所有孤 岛都可通过联通域分析删除。剪 除原则【问孙宇龙要 ref 指向 algorithm / threshold 】【或干脆给 一个伪代码】 subset number reduction 第二步	
4	耗时波动大	(1) ONNX runtime 部署： PyTorch 转为 ONNX 格式，硬件加 速（ 这里我们做了哪些改进？ ） (2) 空间降维（问： 从？维到？ 维 ）（ 怎么降维？有啥 ref？ ）	